

AI MLR Platform - Technical Documentation

Medical-Legal-Regulatory (MLR) Compliance Platform

AI-Powered Document Validation and Compliance Analysis System

Version: 1.0.0

Last Updated: March 2026

Status: Production

Table of Contents

1. Executive Summary
2. System Architecture
3. Technology Stack
4. Backend Service Overview
5. Frontend Application Overview
6. Key Features & Capabilities
7. API Architecture
8. Data Storage & Management
9. Deployment Architecture
10. Security & Configuration

Executive Summary

The AI MLR Platform is a comprehensive compliance analysis system designed for medical-legal-regulatory professionals. It leverages artificial intelligence to automatically validate pharmaceutical and medical documents against regulatory guidelines, identify compliance issues, and provide actionable recommendations.

Core Purpose:

- Automate compliance checking for medical documents
- Provide AI-powered analysis of promotional content
- Manage disclaimer libraries and compliance rules
- Enable real-time chat assistance for compliance questions
- Support design compliance reviews through Figma integration

Target Users:

- Regulatory Affairs Specialists
- Medical-Legal Professionals
- Compliance Officers
- Marketing and Communications Teams
- Quality Assurance Personnel

AI Agents & Intelligent Workflows

What are AI Agents in This Platform?

An AI agent is an intelligent software component that can:

- Understand complex tasks and break them down into steps
- Make decisions about which actions to take
- Use available tools to gather information
- Learn from results and refine decisions
- Interact with external systems autonomously

The AI MLR Platform uses multiple specialized agents to handle different compliance tasks, each with specific knowledge and tools.

Agent Architecture

Core Components of Each Agent

Each agent in the platform consists of:

1. Intelligence Engine - Powered by Google Gemini 2.5 Flash AI model

- Understands compliance requirements
- Analyzes document content
- Makes judgment calls on compliance

2. Tool Arsenal - Specialized functions for specific tasks

- Document analysis tools

- Database search tools
- Calculation tools
- Validation tools

3. **Memory & Context** - Maintains conversation and analysis history

- Previous decisions
- Document processing results
- User feedback

4. **Decision Framework** - Logic for choosing next steps

- Rules for when to use each tool
- Escalation procedures
- Confidence thresholds

LangGraph: Agent Orchestration Engine

What is LangGraph?

LangGraph is a framework that organizes AI agents as directed graphs (flowcharts). Instead of linear sequential processing, it allows agents to:

- Make conditional decisions
- Loop back to refine analysis
- Branch into different workflows
- Combine multiple AI operations

Graph-Based Workflow Benefits

- Agents can dynamically decide next steps
- Complex workflows become manageable
- Failure points are clear and recoverable
- Multiple agents can collaborate on same task

System Architecture

Architecture Overview

The platform follows a modern client-server architecture with clear separation of concerns:

Frontend Layer (Client-Side)

- Angular 15-based Single Page Application
- User interface for all compliance functions
- Real-time data visualization and reporting

- Responsive design for desktop and tablet use

API Gateway

- FastAPI-based REST API server
- Server-Sent Events (SSE) for real-time streaming updates
- Request validation and error handling
- Authentication and authorization

Business Logic Layer

- Document validation engine
- AI-powered compliance analysis
- Chat assistant with knowledge base integration
- Claim review workflow
- Figma design analysis

Data & Integration Layer

- Vector databases for semantic search
- Metadata storage (JSON-based)
- External AI services (Google Gemini, Tavily)
- OCR and text extraction services

Technology Stack

Backend Technologies

Framework & Server

- FastAPI: Modern Python web framework for building REST APIs
- Uvicorn/Gunicorn: ASGI application servers for production deployment
- Python 3.12: Latest stable Python runtime

Artificial Intelligence & LLM

- Google Gemini 2.5 Flash: Primary language model for compliance analysis
- LangGraph: Workflow orchestration engine for multi-step AI processes
- Sentence-Transformers: Deep learning models for semantic embeddings

Vector Search & Storage

- FAISS: Facebook AI Similarity Search for semantic similarity matching
- Multiple specialized vector stores for different content types
- In-memory caching for performance optimization

Document Processing

- Tesseract OCR: Optical Character Recognition for scanned documents
- pypdfium2: High-quality PDF rendering and extraction
- pypdf: PDF manipulation and text extraction
- python-docx, python-pptx: Document format support

Additional Services

- Tavily: Real-time web search integration
- Pydantic: Data validation and schema management
- python-dotenv: Environment configuration management

Frontend Technologies

Framework & UI

- Angular 15: Enterprise-grade SPA framework
- TypeScript 4.9: Type-safe JavaScript development
- Angular Material 15: Pre-built Material Design components
- Bootstrap 5: Responsive layout system

UI Components & Visualization

- ngx-extended-pdf-viewer: Embedded PDF viewing capabilities
- marked: Markdown to HTML rendering
- animate.css: CSS-based animations
- Bootstrap Icons: Icon library

State Management & Networking

- RxJS: Reactive programming with Observables
- Angular HttpClient: HTTP communication with backend
- Server-Sent Events: Real-time streaming from backend

Testing & Quality

- Jasmine: Testing framework
- Karma: Test runner
- Angular Testing Utilities: Testing infrastructure

Infrastructure & Deployment

Containerization

- Docker: Multi-stage containerization for both backend and frontend
- Docker Compose: Orchestration of multiple containers

System Dependencies

- Tesseract OCR: Runtime library
- Poppler: PDF processing utility
- libgl1: Graphics library support

Web Server

- Nginx: Reverse proxy and static file serving for frontend

Backend Service Overview

Core Components

Main Application (main.py)

The entry point of the FastAPI application that:

- Initializes the web framework
- Configures CORS (Cross-Origin Resource Sharing) for frontend communication
- Enables GZip compression for response optimization
- Registers all API routes and endpoints
- Preloads machine learning models at startup

Configuration Module (app/config.py)

Centralized management of:

- File system paths for vector stores and document storage
- Environment variables for API keys and settings
- Global application state that persists across requests

Utility Functions (app/utils.py)

Provides shared functionality:

- JSON file reading/writing operations
- Prompt template loading for AI models
- Text normalization and cleaning
- Common validation functions

API Modules

Embeddings Engine

- Manages machine learning models for semantic understanding
- Converts text to numerical vectors for similarity searches
- Caches models for performance

OCR (Optical Character Recognition)

- Extracts text from PDF images
- Handles scanned documents and image-based content
- Provides fallback when native PDF text extraction fails

PDF Processing

- Extracts text and metadata from PDF documents
- Splits documents into pages
- Handles multi-page document processing

Validation Engine

- Core workflow for compliance checking
- Uses AI to analyze each page against active disclaimers
- Identifies missing, incorrect, or properly positioned disclaimers
- Streams results in real-time to frontend

Chat Assistant

- Conversational AI interface for compliance questions
- Integrates knowledge base search
- Supports web search for up-to-date information
- Maintains conversation history

Voice Processing

- Speech-to-text conversion for voice input
- Text-to-speech for audio responses
- Integrates with chat assistant

Figma Integration

- Connects to Figma API for design file access
- Extracts text from design frames

- Analyzes design compliance
- Provides visual overlays of compliance issues

Claim Review Engine

- Extracts claims from promotional content
- Compares against approved claim library
- References regulatory guidelines
- Generates compliance reports

Dashboard & Analytics

- Aggregates validation statistics
- Caches results for performance
- Provides historical compliance data

Data Flow

Document Upload Process

1. User uploads PDF through frontend
2. Backend receives and stores document
3. PDF pages extracted and analyzed
4. Page metadata stored in application state

Validation Process

1. Frontend triggers validation workflow
2. Backend processes each page sequentially
3. AI analyzes compliance for each page
4. Results streamed to frontend in real-time
5. User sees live updates without page reload

Chat Interaction

1. User enters message in chat interface
2. AI determines if additional information needed
3. System searches knowledge base if required
4. Web search performed for current information if needed

5. Response generated and displayed to user

Frontend Application Overview

Application Structure

Root Module (`app.module.ts`)

- Declares all components and services
- Imports required Angular modules and third-party libraries
- Configures Material Design theme

Routing Configuration (`app-routing.module.ts`)

- Defines navigation paths for all application sections
- Lazy-loads feature modules when needed
- Manages route transitions

Core Features

Dashboard Component

- Displays key performance indicators
- Shows validation success rates
- Presents historical compliance trends
- Provides quick access to recent documents

Document Validation Component

- PDF upload interface
- Real-time validation progress display
- Results table with detailed findings
- Options to view original PDF and compliance details

Chat Assistant Component

- Text messaging interface
- Voice recording capability
- Conversation history display
- Markdown rendering for formatted responses

Disclaimer Library Component

- View all active disclaimers
- Add new disclaimers
- Manage disclaimer categories

- Activate/deactivate disclaimers

Claim Library Component

- Approved claims database
- Search and filter capabilities
- Evidence level tracking
- Clinical category organization

Figma Review Component

- Figma file ingestion interface
- Visual compliance overlay display
- Text extraction results
- Compliance issue highlighting

Claim Intelligence Component

- Promotional content upload
- Reference guideline input
- Comprehensive compliance analysis
- Remediation recommendations

Service Architecture

Document Service

- Handles PDF upload and retrieval
- Manages document state
- Communicates with upload endpoints

Validation Service

- Triggers validation workflows
- Subscribes to real-time updates via SSE
- Manages validation history

Chat Service

- Sends messages to backend
- Manages voice audio files
- Maintains chat context

Disclaimer Service

- CRUD operations for disclaimers
- Manages active disclaimer list
- Syncs with backend

Claim Service

- Submits content for review
- Retrieves compliance reports
- Manages claim library

Figma Service

- Fetches Figma designs
- Manages design analysis results
- Coordinates visual overlays

User Interface

Navigation

- Sidebar menu for section access
- Header with branding and user info
- Footer with version information

Layouts

- Dashboard: KPI cards and charts
- Validation: PDF viewer + results table
- Chat: Message interface + conversation history
- Libraries: Searchable tables with CRUD actions

Key Features & Capabilities

Document Validation Workflow

- Page-by-page compliance analysis
- Automatic disclaimer identification
- Missing/incorrect disclaimer detection
- Real-time progress streaming
- Multi-page document support
- OCR fallback for scanned documents

Agent Role in Validation:

The Validation Agent analyzes each PDF page and determines compliance status. It:

- Reads document content (text or OCR from images)
- Searches the disclaimer database for relevant disclaimers
- Compares page content against each disclaimer
- Decides if a disclaimer is missing, incorrect, or properly placed

- Assigns confidence scores to decisions
- Streams results in real-time as it processes each page

AI-Powered Chat Assistant

- Conversational compliance guidance
- Knowledge base integration
- Real-time web search capability
- Voice input/output support
- Multi-turn conversation history
- Source attribution

Agent Role in Chat Assistant:

The Chat Agent is a conversational AI that answers compliance questions. It:

- Listens to user questions
- Decides what information it needs to answer accurately
- Has three tool choices:

1. Search internal knowledge base for MLR guidelines
2. Search the web for current regulatory updates
3. Query the disclaimer library for specific disclaimers

- Combines information from multiple sources
- Generates a comprehensive, cited response
- Maintains conversation history for context
- Can clarify or ask follow-up questions

Claim Compliance Review

- Promotional content analysis
- Claim extraction and evaluation
- Library matching with semantic search
- Reference guideline comparison
- Detailed compliance recommendations

Agent Role in Claim Review:

The Claim Review Agent evaluates promotional claims for compliance. It:

- Extracts individual claims from promotional materials
- For each claim, searches the approved claim library
- Evaluates the evidence level of the claim
- Compares against reference guidelines and regulations
- Identifies if claims match approved versions

- Flags unsupported or risky claims
- Generates detailed compliance recommendations
- Suggests remediation for non-compliant claims

Figma Design Analysis

- Design file integration
- Text node extraction
- Visual compliance overlay
- Frame-by-frame analysis
- Compliance issue highlighting

Disclaimer Management

- Centralized disclaimer library
- Category organization
- Activation/deactivation workflow
- Search capabilities
- Metadata tracking

Dashboard Analytics

- Compliance rate metrics
- Processing statistics
- Historical data tracking
- Performance indicators

API Architecture

Specialized AI Agents Used in the Platform

Agent Orchestration with LangGraph

What Happens Behind the Scenes

When an agent processes a task, LangGraph orchestrates the workflow:

1. **Initialization** - Agent receives input and initializes state
2. **Tool Selection** - Agent decides which tools to use
3. **Tool Execution** - Selected tools run and return results
4. **Result Processing** - Agent analyzes tool results
5. **Decision Making** - Agent decides next step:
 - Use another tool?

- Refine previous analysis?
- Generate final answer?

6. **Output Generation** - Agent produces response

7. **Streaming** - Results sent to frontend in real-time

Conditional Logic Flow

Agent Receives Input

↓

Does agent have enough info?

├ No: Select tool to get more info

| ↓

| Execute Tool

| ↓

| Is result satisfactory?

| ├── No: Try different tool

| └ Yes: Continue

|

└ Yes: Analyze & Generate Response

↓

Provide Answer to User

Specialized AI Agents Used in the Platform

The MLR Platform employs four main AI agents, each specialized for specific compliance tasks:

1. *Validation Agent*

Purpose: Analyze PDF documents page-by-page for compliance

Workflow Process:

1. Receives uploaded PDF document
2. Extracts text from each page (or uses OCR for scanned pages)
3. For each page, searches disclaimer database for relevant disclaimers
4. Analyzes page content against active disclaimers
5. Determines compliance status:
 - Does page have correct disclaimer? ✓
 - Does page have wrong disclaimer? ✗
 - Is a required disclaimer missing? ⚠
6. Streams results to frontend in real-time
7. Generates final compliance report

Key Decision Points:

- Should this disclaimer be on this page?
- Is the disclaimer correctly positioned?
- What is the confidence level of this decision?

Tools Available:

- Text extraction from PDFs
- OCR for image-based content
- Disclaimer database search
- Similarity matching with embeddings

2. Chat Agent

Purpose: Provide conversational compliance guidance

Workflow Process:

1. Receives user question about compliance
2. Analyzes the question to understand what information is needed
3. Decides which tools to use:
 - **Option A:** Search knowledge base for regulatory guidelines
 - **Option B:** Search web for current regulatory updates
 - **Option C:** Search disclaimer library for specific disclaimers
 - **Option D:** Provide answer from training knowledge
4. Gathers information from selected sources

5. Synthesizes answer with citations
6. Presents response to user
7. Maintains conversation history for multi-turn discussions

Key Decision Points:

- Does this answer require external information?
- Should I search internal knowledge or the web?
- Do I have enough information to respond?

Tools Available:

- Internal knowledge base search
- Web search via Tavily
- Disclaimer library lookup
- Conversation history management

3. Claim Review Agent

Purpose: Evaluate promotional claims for compliance

Workflow Process:

1. Receives promotional content (PDF, JSON, or text)
2. Extracts individual claims from the content
3. For each claim:
 - Searches approved claim library for similar claims
 - Retrieves evidence levels and clinical categories
 - Cross-references with regulatory guidelines
 - Compares against reference CSVs
4. Classifies each claim:
 - Approved and compliant ✓
 - Similar to approved with modifications ≈
 - Not found in library ✗
 - High-risk or unsupported ⚠
5. Generates remediation recommendations
6. Creates compliance assessment report

Key Decision Points:

- Is this claim substantiated by evidence?
- Is the claim similar to an approved version?
- What evidence level is required?
- Should this claim be flagged for review?

Tools Available:

- Content parsing and extraction
- Claim library semantic search
- Reference guideline matching
- Evidence evaluation

4. Design Analysis Agent (Figma)

Purpose: Analyze design files for compliance text and issues

Workflow Process:

1. Receives Figma file key or URL
2. Fetches design file via Figma API
3. Extracts all text nodes from design frames
4. Analyzes text content for compliance issues
5. Checks extracted text against disclaimers
6. Identifies design elements with text
7. Creates visual overlay marking compliance issues
8. Generates design-specific compliance report

Key Decision Points:

- Is this text visible to end users?
- Is this text part of the marketing claim?
- Should a disclaimer appear near this text?

Tools Available:

- Figma API integration
- Text node extraction
- Visual coordinate tracking
- Compliance overlay generation

How Agents Make Decisions

Multi-Step Decision Making Process

1. Understand the Task

- Agent reads input (document, question, content)
- Identifies the type of compliance analysis needed
- Recalls relevant rules and guidelines

2. Gather Information

- Agent decides what additional information is needed
- Uses available tools to search databases
- Collects relevant context and examples

3. Analyze and Compare

- Agent compares input against known guidelines
- Evaluates multiple possible answers
- Calculates confidence scores

4. Make Decision

- Agent selects the most appropriate answer
- Provides reasoning and evidence
- Flags areas of uncertainty

5. Explain and Stream

- Agent presents findings to user
- Provides citations and sources
- Streams results in real-time for better UX

Confidence Scoring

Each agent decision includes a confidence score (0-100%) indicating:

- How certain the agent is about the decision
- Whether additional human review is recommended
- Whether the decision requires escalation

Agent Tools & Integrations

Validation Agent Tools:

- PDF text extraction engine
- OCR for scanned documents
- FAISS vector database search

- Disclaimer comparison engine

Chat Agent Tools:

- Knowledge base semantic search
- Tavily web search engine
- Disclaimer database query
- Conversation memory storage

Claim Review Agent Tools:

- Content extraction parser
- Claim library search engine
- Regulatory guideline matcher
- Evidence level evaluator

Figma Agent Tools:

- Figma REST API client
- Text node parser
- Visual coordinate mapper
- Compliance overlay renderer

Agent Learning & Improvement

Feedback Loop

- User feedback on agent decisions improves accuracy
- Incorrect decisions are logged and analyzed
- System learns from compliance mistakes
- Guidelines are updated based on regulatory changes

Continuous Refinement

- Prompt optimization for better decision-making
- Tool improvements for faster information retrieval
- Confidence threshold adjustments
- New tool additions for emerging compliance needs

API Architecture

API Design Pattern

- RESTful API design principles
- JSON request/response format
- Standard HTTP status codes

- Error handling with descriptive messages

Main Endpoint Categories

Health & Status

- Service health checks
- Version information

Document Management

- PDF upload endpoint
- Disclaimer upload endpoint
- Document retrieval

Validation

- Validation workflow trigger
- Real-time result streaming
- Validation history retrieval

Chat

- Message submission endpoint
- Conversation history retrieval
- Voice chat endpoint

Compliance Review

- Claim review submission
- Results retrieval
- Report generation

Design Analysis

- Figma file ingestion
- Compliance overlay retrieval
- Visual analysis results

Library Management

- Disclaimer CRUD operations
- Claim library management
- Active set management

Analytics

- Dashboard statistics

- Historical compliance data
- Trend analysis

Real-Time Communication

- Server-Sent Events (SSE) for streaming validation progress
- One-way streaming from server to client
- No connection persistence needed
- Automatic reconnection handling

Data Storage & Management

Vector Store Architecture

Purpose of Vector Stores

Vector stores enable semantic similarity searching by converting text into mathematical vectors. The system uses FAISS (Facebook AI Similarity Search) for efficient nearest-neighbor retrieval.

Multiple Specialized Stores

- **Disclaimer Store:** Active regulatory disclaimers and their embeddings
- **Claim Library Store:** Approved claims and evidence
- **Knowledge Base Store:** General MLR compliance information
- **Feedback Store:** User feedback and patterns
- **Dashboard Store:** Analytics and trending data

Metadata Management

- Stores are paired with JSON metadata files
- Metadata includes: ID, text, category, timestamps, status
- Enables filtering and administration
- Separates AI vectors from human-readable information

Data Organization

Disclaimer Management

- Centralized index of all disclaimers
- Metadata tracking creation dates and categories
- Active/inactive status for soft-deletion
- Linked to vector embeddings for semantic search

Claim Library

- Approved claims with evidence levels
- Clinical category classification

- Regulatory references
- Approval workflow tracking

Session & State Data

- Temporary storage of uploaded documents
- Processing results cached during validation
- Real-time progress tracking
- Automatic cleanup after completion

Deployment Architecture

Containerization Strategy

Multi-Stage Docker Builds

- Separate build and runtime stages for efficiency
- Reduces final image size and security attack surface
- Build stage contains only development dependencies
- Runtime stage contains only necessary libraries

Backend Container

- Based on Python 3.12-slim image
- Includes Tesseract OCR and Poppler utilities
- Exposes port 8080
- Runs FastAPI application

Frontend Container

- Based on Node 20 Alpine for building
- Based on Nginx Alpine for serving
- Serves compiled Angular application
- Proxy configuration for API calls

Orchestration

Docker Compose Configuration

- Defines backend and frontend services
- Network isolation and communication
- Environment variable configuration
- Volume mounting for persistent data

Service Communication

- Backend accessible at backend:8080 within network
- Frontend routes API calls through nginx proxy
- Services communicate via internal Docker network

Production Considerations

- Multiple backend instances for load balancing
- Persistent volume storage for vector stores
- Environment-specific configuration
- Logging and monitoring setup

Security & Configuration

Environment Configuration

Required Environment Variables

- GEMINI_API_KEY: Google API credentials
- TAVILY_API_KEY: Web search API credentials
- EMBEDDING_MODEL: HuggingFace model selection
- PORT: Application port configuration

Sensitive Data Management

- Environment variables for secrets (not hardcoded)
- .env files excluded from version control
- Key rotation procedures recommended
- Production secrets manager integration

Security Best Practices

API Security

- CORS configuration for allowed origins
- Input validation on all endpoints
- Rate limiting on sensitive endpoints
- Request size limits

Data Protection

- HTTPS/TLS for production deployment
- Encrypted communication channels
- Secure token handling
- Data encryption at rest

Access Control

- Role-based access control capability
- API key management
- Audit logging
- Session management

CORS Configuration

- Currently allows all origins for development
- Production should restrict to specific domains
- Credentials handling for cross-site requests

Development & Operations

Local Development Setup

Backend Development

- Python virtual environment for isolation
- Uvicorn development server with auto-reload
- Automatic API documentation at /docs
- Hot module replacement for code changes

Frontend Development

- npm for dependency management
- Angular development server
- Hot module replacement for instant updates
- Chrome DevTools integration

Deployment Process

Build Pipeline

- Code compilation and bundling
- Dependency resolution
- Container image creation
- Registry push for distribution

Release Process

- Version tagging and release notes
- Container image versioning
- Changelog maintenance
- Documentation updates

Monitoring & Maintenance

Health Monitoring

- Service health check endpoints
- Application performance metrics
- Error logging and tracking
- Resource utilization monitoring

Backup & Recovery

- Vector store backups
- Metadata backup procedures
- Recovery testing
- Disaster recovery planning

System Integration

External Services

Google Gemini API

- Primary AI model for compliance analysis
- Speech-to-text and text-to-speech capabilities
- Requires API key and account setup
- Rate limiting considerations

Tavily Search API

- Real-time web search integration
- Current regulatory updates retrieval
- Evidence gathering for compliance

Figma API

- Design file access and extraction
- Text node analysis
- Frame-level processing
- Visual coordinate tracking

HuggingFace Models

- Sentence-Transformers for embeddings
- Pre-trained semantic understanding
- Automatic model download and caching

Performance Considerations

Optimization Strategies

Caching

- Embeddings model caching
- Vector store pre-loading
- API response caching
- TTL-based cache invalidation

Async Processing

- Non-blocking API endpoints
- Concurrent page processing
- Real-time streaming for user feedback
- Background task processing

Database Optimization

- FAISS indexing for fast search
- Metadata JSON for quick access
- Strategic data partitioning
- Index rebuilding procedures

Scalability

Horizontal Scaling

- Multiple backend instances
- Load balancer configuration
- Stateless API design
- Shared vector store access

Performance Monitoring

- Response time tracking
- Query performance analysis
- Resource utilization metrics
- Bottleneck identification

Conclusion

The AI MLR Platform represents a comprehensive solution for medical-legal-regulatory compliance analysis. By combining advanced AI capabilities with intuitive user interfaces, the system enables

organizations to automate compliance checking, reduce human error, and maintain regulatory adherence efficiently.

Key Strengths:

- Automated compliance analysis at scale
- Real-time feedback and streaming updates
- Multiple compliance review methodologies
- Integration with modern design workflows
- Scalable and containerized architecture

Technology Highlights:

- State-of-the-art AI models (Google Gemini)
- Semantic search capabilities (FAISS)
- Production-ready architecture
- Responsive web interface
- RESTful API design

Document Version: 1.0.0

Last Updated: March 30, 2026

Status: Complete

Maintainer: AI MLR Development Team